

Chapter 1

Huffman Compression

```

1  /*
2  HUFF2412 File Format Specification (Encoder v1 / Decoder v1)
3
4  Overview
5  -----
6  This program writes and reads a Huffman-compressed file format. The output file is:
7
8      [HEADER (byte-aligned fields + bit-packed tree)] [PAYLOAD (bit-packed codes)]
9
10 "Payload" = the compressed bitstream for the original file contents, produced by
11 replacing each input byte with its Huffman code bits.
12
13 IMPORTANT: Multi-byte integers are written using the platform's native in-memory
14 representation (typically little-endian). This format is therefore NOT portable
15 across architectures with different endianness unless an explicit byte order is used.
16
17 Header Layout (in order)
18 -----
19 1) Magic (8 bytes)
20   ASCII bytes: 'H' 'U' 'F' 'F' '2' '4' '1' '2'
21   Used to identify files produced by this compressor.
22
23 2) Original Size (uint64_t, 8 bytes)
24   Exact number of bytes in the original uncompressed input file.
25   Decoder MUST stop after producing exactly this many bytes (ignores any final padding).
26
27 3) Stored Filename Length (uint16_t, 2 bytes)
28   Length in bytes of the stored filename string.
29
30 4) Stored Filename (nameLen bytes)
31   Raw bytes of the input file base name (includes extension), not NUL-terminated.
32   Example: "photo.png" or "archive.tar.gz".
33
34   Decoder behavior:
35   - The decoder attempts to write output to this filename in the current directory.
36   - If the file already exists, the decoder prompts the user before overwriting.
37
38 5) Tree Bit Length (uint32_t, 4 bytes)
39   Number of bits used to encode the serialized Huffman tree that immediately follows.
40   This allows the decoder to know exactly where the tree ends and where the payload begins.
41
42 6) Serialized Huffman Tree (treeBitsLen bits, bit-packed)
43   Preorder serialization (MSB-first bit order within each byte):
44   - Leaf node:
45     write bit 1
46     then write 8 bits of the leaf byte value
47     Total = 9 bits per leaf.
48

```

```

49   - Internal node:
50     write bit 0
51     then serialize left subtree
52     then serialize right subtree
53     Total = 1 bit + left subtree bits + right subtree bits.
54
55 7) Tree Padding (0..7 bits)
56   After the tree bits are written, the encoder pads with 0 bits up to the next
57   byte boundary so that the payload starts on a byte boundary.
58   Decoder should discard these padding bits by aligning to the next byte boundary.
59
60 Payload Layout
61 -----
62 8) Huffman Payload Bits (variable length, bit-packed)
63   For each input byte, the encoder writes the corresponding Huffman code bits
64   (from the code table built from the Huffman tree).
65   Bit order is MSB-first within output bytes, matching the BitWriter/BitReader.
66
67 9) Final Padding (0..7 bits)
68   After encoding all symbols, the encoder pads the last output byte with 0 bits
69   (if needed) and flushes it.
70   The decoder ignores any remaining padding bits because it stops after outputting
71   exactly Original Size bytes.
72
73 Notes / Constraints / Error Handling
74 -----
75 - Empty input files are currently rejected by the encoder.
76 - Codes are limited to 64 bits in this implementation (ERR_RECURSION_DEPTH if exceeded).
77 - Decoder detects invalid/corrupt streams as ERR_FORMAT (bad magic, invalid tree encoding,
78   impossible traversal, mismatched treeBitsLen consumption, etc.).
79 - I/O failures while reading return ERR_READ; I/O failures while writing return ERR_WRITE.
80 - If the user declines overwrite when output already exists, the decoder returns ERR_ABORTED.
81 */
82
83 #include <stddef.h>
84 #include <stdint.h>
85 #include <stdio.h>
86 #include <inttypes.h>
87 #include <stdlib.h>
88 #include <string.h>
89 #include <limits.h>
90
91 #define BUFFER_SIZE 8192
92 #define FREQ_SIZE 256
93
94 /**
95  * Enum for error handling
96  */
97
98 typedef enum {
99     SUCCESS,
100     ERR_MEMORY,
101     ERR_FILE,
102     ERR_READ,
103     ERR_HEAP_CAPACITY_FULL,
104     ERR_RECURSION_DEPTH,
105     ERR_WRITE,
106     ERR_FORMAT,
107     ERR_ABORTED
108 } Status;
109
110 /**
111  * Reads the input file and generates frequency table for the characters
112  *
113  * @param filename file to be read
114  * @param freq pointer to freq table
115  * @param totalBytes pointer to variable totalbytes which stores the total bytes read here
116  */

```

```

117  * @return SUCCESS if file was read, table was generated and function returned cleanly
118  */
119
120  /*===== READ FILE =====*/
121
122  Status readFile(const char* filename, uint64_t **freq, uint64_t *totalBytes) {
123      FILE *f = fopen(filename, "rb");
124
125      if(!f) {
126          return ERR_FILE;
127      }
128
129      *freq = (uint64_t*) calloc(FREQ_SIZE, sizeof(uint64_t));
130      if(!*freq) {
131          fclose(f);
132          return ERR_MEMORY;
133      }
134
135      uint8_t *buffer = (uint8_t*) malloc(BUFFER_SIZE);
136      if(!buffer) {
137          free(*freq);
138          *freq = NULL;
139          fclose(f);
140          return ERR_MEMORY;
141      }
142
143      size_t bytesRead = 0;
144      *totalBytes = 0;
145
146      while((bytesRead = fread(buffer, 1, BUFFER_SIZE, f)) > 0) {
147          *totalBytes += bytesRead;
148
149          for(size_t i=0; i<bytesRead; i++) {
150              (*freq)[buffer[i]]++;
151          }
152      }
153
154      if(ferror(f)) {
155          free(*freq);
156          *freq = NULL;
157          free(buffer);
158          fclose(f);
159          return ERR_READ;
160      }
161
162      free(buffer);
163      fclose(f);
164      return SUCCESS;
165  }
166
167  /*===== NODE =====*/
168
169  typedef struct Node {
170      uint8_t byte;
171      uint64_t freq;
172      struct Node *left;
173      struct Node *right;
174  } Node;
175
176  /**
177   * Creates a newnode and updates it's byte, freq, left and right fields
178   * with appropriate values
179   */
180
181  Status createNode(Node **newNode, uint8_t byte, uint64_t freq, Node *left, Node *right) {
182      *newNode = (Node*) malloc(sizeof(Node));
183      if(!*newNode) {
184          return ERR_MEMORY;

```

```

185  }
186
187      (*newNode)->byte = byte;
188      (*newNode)->freq = freq;
189      (*newNode)->left = left;
190      (*newNode)->right = right;
191
192      return SUCCESS;
193  }
194
195  /**
196   * Frees the tree structure recursively
197   */
198
199  void freeTree(Node **root) {
200      if(!root || !*root) return;
201      freeTree(&(*root)->left);
202      freeTree(&(*root)->right);
203      free(*root);
204      *root = NULL;
205  }
206
207  /*===== HEAP =====*/
208
209  typedef struct {
210      Node **arr;
211      size_t size;
212      size_t capacity;
213  } MinHeap;
214
215  void swap(Node **a, Node **b) {
216      Node *tmp = *a;
217      *a = *b;
218      *b = tmp;
219  }
220
221  /**
222   * Implements the heapify down algorithm recursively
223   */
224
225  void heapifyDown(MinHeap *h, size_t i) {
226      size_t smallest = i;
227      size_t left = 2*i + 1;
228      size_t right = 2*i + 2;
229
230      if(left < h->size && h->arr[left]->freq < h->arr[smallest]->freq) {
231          smallest = left;
232      }
233
234      if(right < h->size && h->arr[right]->freq < h->arr[smallest]->freq) {
235          smallest = right;
236      }
237
238      if(smallest != i) {
239          swap(&h->arr[i], &h->arr[smallest]);
240          heapifyDown(h, smallest);
241      }
242  }
243
244  /**
245   * Implements the heapify up algorithm recursively
246   */
247
248  void heapifyUp(MinHeap *h, size_t i) {
249      if(i == 0) {
250          return;
251      }
252

```

```

253     size_t parent = (i - 1)/2;
254
255     if(h->arr[i]->freq < h->arr[parent]->freq) {
256         swap(&h->arr[i], &h->arr[parent]);
257         heapifyUp(h, parent);
258     }
259 }
260
261 /**
262  * Insert a node in the min heap and calls heapify up
263  */
264
265 Status heapInsert(MinHeap *h, Node *node) {
266     if(h->size ≥ h->capacity) return ERR_HEAP_CAPACITY_FULL;
267
268     h->arr[h->size] = node;
269     heapifyUp(h, h->size);
270     h->size++;
271
272     return SUCCESS;
273 }
274
275 /**
276  * Extracts the root node from the heap and calls heapify down
277  */
278
279 Node* extractMin(MinHeap *h) {
280     if(h->size == 0) return NULL;
281
282     Node *min = h->arr[0];
283     h->arr[0] = h->arr[h->size-1];
284     h->size--;
285
286     heapifyDown(h, 0);
287     return min;
288 }
289
290 /**
291  * Frees the heap structure
292  */
293
294 void destroyHeapStruct(MinHeap **h) {
295     if(!h || !*h) return;
296     free((*h)->arr);
297     free(*h);
298     *h = NULL;
299 }
300
301 /**
302  * Frees the heap structure along with all the nodes inside
303  * via calling freeTree()
304  */
305
306 * It is to be used when tree is not formed and heap owns all the nodes
307 */
308
309 void destroyHeapWithNodes(MinHeap **h) {
310     if(!h || !*h) return;
311
312     if((*h)->arr) {
313         for(size_t i=0; i<(*h)->size; i++) {
314             freeTree(&(*h)->arr[i]);
315         }
316
317         free((*h)->arr);
318         free(*h);
319         *h = NULL;
320     }

```

```

321
322 /*===== BUILD HEAP =====*/
323
324 /**
325  * Builds the heap structure using the freq table
326  */
327
328 Status buildHeap(MinHeap **h, const uint64_t *freq) {
329     *h = NULL;
330     Status status = SUCCESS;
331
332     *h = (MinHeap*) malloc(sizeof(MinHeap));
333     if(!*h) {
334         return ERR_MEMORY;
335     }
336
337     (*h)->capacity = FREQ_SIZE;
338     (*h)->size = 0;
339     (*h)->arr = (Node**) malloc((*h)->capacity * sizeof(Node*));
340     if(!(*h)->arr) {
341         free(*h);
342         return ERR_MEMORY;
343     }
344
345     for(size_t i=0; i<FREQ_SIZE; i++) {
346         if(freq[i] > 0) {
347             Node *newNode = NULL;
348
349             status = createNode(&newNode, (uint8_t) i, freq[i], NULL, NULL);
350             if(status ≠ SUCCESS) goto cleanup;
351
352             status = heapInsert(*h, newNode);
353             if(status ≠ SUCCESS) {
354                 free(newNode);
355                 goto cleanup;
356             }
357         }
358     }
359
360     return SUCCESS;
361
362 cleanup:
363     destroyHeapWithNodes(h);
364     return status;
365 }
366
367 /*===== BUILD HUFFMAN TREE =====*/
368
369 /**
370  * Builds the huffman tree using the minheap generated by buildHeap()
371  */
372
373 Status buildHuffmanTree(MinHeap *h, Node **root) {
374     Status status = SUCCESS;
375     *root = NULL;
376
377     if(!h || h->size == 0) {
378         return SUCCESS;
379     }
380
381     while(h->size > 1) {
382         Node *a = extractMin(h);
383         Node *b = extractMin(h);
384
385         if(!a || !b) {
386             freeTree(&a);
387             freeTree(&b);
388             return ERR_FILE;

```

```

389     }
390
391     Node *merged = NULL;
392
393     status = createNode(&merged, 0, a->freq + b->freq, a, b);
394     if(status != SUCCESS) {
395         freeTree(&a);
396         freeTree(&b);
397         return status;
398     }
399
400     status = heapInsert(h, merged);
401     if(status != SUCCESS) {
402         freeTree(&merged);
403         return status;
404     }
405 }
406
407 *root = extractMin(h);
408 h->size = 0;
409 return SUCCESS;
410 }
411
412 /**
413  * Recursively computes the number of bits used to store the tree in the header
414  * */
415
416 uint64_t treeBitLength(const Node *root) {
417     if(!root) return 0;
418
419     if(!root->left && !root->right) {
420         return 1+8;
421     }
422
423     return 1 + treeBitLength(root->left) + treeBitLength(root->right);
424 }
425
426 /*===== GENERATE HUFFMAN CODES =====*/
427
428 typedef struct {
429     uint64_t bits;
430     uint8_t length;
431 } Code;
432
433 Status initTable(Code **table) {
434     *table = (Code*) calloc(256, sizeof(Code));
435     if(!*table) return ERR_MEMORY;
436     return SUCCESS;
437 }
438
439 /**
440  * Recursively builds the huffman codes for the characters in the frequency table
441  * via traversing the tree, assigning 0 when taking the left branch
442  * and 1 when taking the right branch
443  *
444  * It handles the edge case in which a file may contain a single character by
445  * a single bit code of length 1
446  *
447  * In this version characters whose bit pattern exceed 64 bits aren't supported
448  * as uint64_t is unsigned 64 bit integer which cannot store more than 64 bits
449  * */
450
451 Status buildCodes(const Node *root, Code *table, uint64_t code, uint8_t depth) {
452     if(!root) return SUCCESS;
453
454     if(depth ≥ 64) {
455         return ERR_RECURSION_DEPTH;
456     }

```

```

457
458 // LEAF
459 if(!root->left && !root->right) {
460     // single symbol case
461     if(depth == 0) {
462         table[root->byte].bits = 0;
463         table[root->byte].length = 1;
464     }
465
466     else {
467         table[root->byte].bits = code;
468         table[root->byte].length = depth;
469     }
470
471     return SUCCESS;
472 }
473
474 Status s;
475
476 // go left -> 0
477 s = buildCodes(root->left, table, (code << 1) | 0, depth+1);
478 if(s != SUCCESS) return s;
479
480 // go right -> 1
481 s = buildCodes(root->right, table, (code << 1) | 1, depth+1);
482 if(s != SUCCESS) return s;
483
484 return SUCCESS;
485 }
486
487 /*===== BIT WRITER AND UTILS =====*/
488
489 typedef struct {
490     FILE *f;
491     uint8_t buffer;
492     uint8_t bitCount;
493 } BitWriter;
494
495 /**
496  * Initializes BitWriter structure and opens the output file
497  *
498  * buffer and bitcount are initialized to 0
499  * */
500
501 Status initBitWriter(BitWriter *bw, const char *filename) {
502     bw->f = fopen(filename, "wb");
503     if(!bw->f) {
504         return ERR_FILE;
505     }
506
507     bw->buffer = bw->bitCount = 0;
508     return SUCCESS;
509 }
510
511 /**
512  * Writes the bits into buffer first and packs them into a byte
513  * Once the buffer is full of 8 bits it is flushed to the file
514  *
515  * Bits are written in MSB first order
516  * */
517
518 Status writeBits(BitWriter *bw, uint64_t bits, uint8_t length) {
519     for(int i=length-1; i≥0; i--) {
520         uint8_t bit = (uint8_t) ((bits >> i) & 1);
521
522         bw->buffer = (uint8_t) ((bw->buffer << 1) | bit);
523         bw->bitCount++;
524     }

```

```

525     if(bw->bitCount == 0) {
526         if(fwrite(&bw->buffer, 1, 1, bw->f) != 1) return ERR_WRITE;
527
528         bw->buffer = 0;
529         bw->bitCount = 0;
530     }
531 }
532
533 return SUCCESS;
534 }
535
536 /**
537  * At the end the file may have less than 8 bits
538  *
539  * To handle this we write the bits following MSB first ordering
540  * and then pad the remaining bits to 0 and then flush to the file
541  */
542
543 Status writePaddedBits(BitWriter *bw) {
544     // pads leftover bits with 0s and writes them to file
545     if(bw->bitCount == 0) return SUCCESS;
546     bw->buffer = (uint8_t) (bw->buffer << (8-bw->bitCount));
547
548     if(fwrite(&bw->buffer, 1, 1, bw->f) != 1) return ERR_WRITE;
549
550     bw->buffer = bw->bitCount = 0;
551     return SUCCESS;
552 }
553
554 Status closeBitWriter(BitWriter *bw) {
555     if(!bw || !bw->f) return SUCCESS;
556     if(fclose(bw->f) != 0) return ERR_FILE;
557     bw->f = NULL;
558     return SUCCESS;
559 }
560
561 /**
562  * Utility function to extract the filename from full path
563  */
564
565 const char* baseName(const char *path) {
566     if(!path) return "";
567
568     const char *last = path;
569     for(const char *p = path; *p; p++) {
570         if(*p == '/' || *p == '\\') last = p + 1;
571     }
572     return last;
573 }
574
575 /*===== HEADER =====*/
576
577 /**
578  * Writes the huffman tree to the compressed file using preorder traversal
579  *
580  * Internal nodes are written as 0 bit followed by the left subtree and the right subtree
581  *
582  * Leaf nodes are written as 1 bit followed by 8 bit byte value
583  */
584
585 Status writeTree(const Node *root, BitWriter *bw) {
586     if(!root) return ERR_FILE;
587
588     Status s;
589
590     // leaf
591     if(!root->left && !root->right) {
592         s = writeBits(bw, 1, 1);

```

```

593     if(s != SUCCESS) return s;
594
595     s = writeBits(bw, root->byte, 8);
596     if(s != SUCCESS) return s;
597
598     return SUCCESS;
599 }
600
601 // internal
602 s = writeBits(bw, 0, 1);
603 if(s != SUCCESS) return s;
604
605 s = writeTree(root->left, bw);
606 if(s != SUCCESS) return s;
607
608 s = writeTree(root->right, bw);
609 if(s != SUCCESS) return s;
610
611 return SUCCESS;
612 }
613
614 /**
615  * Writes the header in the output file that will help the decoder
616  * to reconstruct the original bit pattern of the file
617  *
618  * The function writes the following in the header:
619  * MAGIC BYTES = 'H','U','F','F','2','4','1','2'
620  *
621  * The original file size which helps the decoder to know after how many bytes it has to stop
622  *
623  * Size of the original filename
624  *
625  * The original filename which is given to the uncompressed file by the decoder
626  *
627  * Length of the huffman tree which is embedded into the header
628  *
629  * The huffman tree which is flattened into bits and then written in header
630  * and will later be reconstructed by the decoder
631  *
632  * Also any remaining bits are padded to zeroes so that there is no
633  * ambiguity left for the decoder
634  */
635
636 Status writeHeader(BitWriter *bw, uint64_t originalSize, const Node *root, const char* inputPath) {
637     const uint8_t magic[] = {'H', 'U', 'F', 'F', '2', '4', '1', '2'};
638
639     if(fwrite(magic, 1, 8, bw->f) != 8) return ERR_WRITE;
640
641     if(fwrite(&originalSize, sizeof(originalSize), 1, bw->f) != 1) return ERR_WRITE;
642
643     const char *name = baseName(inputPath);
644     size_t n = strlen(name);
645     if(n > UINT16_MAX) return ERR_FILE;
646
647     uint16_t nameLen = (uint16_t) n;
648
649     if(fwrite(&nameLen, sizeof(nameLen), 1, bw->f) != 1) return ERR_WRITE;
650     if(nameLen > 0) {
651         if(fwrite(name, 1, nameLen, bw->f) != nameLen) return ERR_WRITE;
652     }
653
654     uint64_t treeBits64 = treeBitLength(root);
655     if(treeBits64 > UINT32_MAX) return ERR_FILE;
656
657     uint32_t treeBitsLen = (uint32_t) treeBits64;
658     if(fwrite(&treeBitsLen, sizeof(treeBitsLen), 1, bw->f) != 1) return ERR_WRITE;
659
660     Status s = writeTree(root, bw);

```

```

661     if(s != SUCCESS) return s;
662
663     s = writePaddedBits(bw);
664     if(s != SUCCESS) return s;
665
666     return SUCCESS;
667 }
668
669 /*===== ENCODER =====*/
670
671 /**
672  * First header is written in the output file
673  *
674  * Then the input file is re-read in chunks of 8192 bytes and
675  * simultaneously output file is written bit by bit using the bitwriter
676  *
677  * Then at last any remaining bits are padded with 0
678  */
679
680 Status encodeFile(const char *inName, const char *outName, const Code *table, uint64_t
originalSize, const Node *root) {
681     Status s = SUCCESS;
682     FILE *in = NULL;
683     uint8_t *buffer = NULL;
684     BitWriter bw = {0};
685
686     in = fopen(inName, "rb");
687     if(!in) return ERR_FILE;
688
689     s = initBitWriter(&bw, outName);
690     if(s != SUCCESS) goto cleanup;
691
692     buffer = (uint8_t*) malloc(BUFFER_SIZE);
693     if(!buffer) {
694         s = ERR_MEMORY;
695         goto cleanup;
696     }
697
698     s = writeHeader(&bw, originalSize, root, inName);
699     if(s != SUCCESS) goto cleanup;
700
701     size_t bytesRead = 0;
702
703     while((bytesRead = fread(buffer, 1, BUFFER_SIZE, in)) > 0) {
704         for(size_t i=0; i<bytesRead; i++) {
705             uint8_t byte = buffer[i];
706
707             if(table[byte].length == 0) {
708                 s = ERR_FILE;
709                 goto cleanup;
710             }
711
712             s = writeBits(&bw, table[byte].bits, table[byte].length);
713             if(s != SUCCESS) goto cleanup;
714         }
715     }
716
717     if(ferror(in)) {
718         s = ERR_READ;
719         goto cleanup;
720     }
721
722     s = writePaddedBits(&bw);
723     if(s != SUCCESS) goto cleanup;
724
725     cleanup:
726     if(in) fclose(in);
727

```

```

728     Status cs = closeBitWriter(&bw);
729     if(s == SUCCESS && cs != SUCCESS) s = cs;
730
731     free(buffer);
732     return s;
733 }
734
735 /*===== HEADER READER =====*/
736
737 typedef struct {
738     uint64_t originalSize;
739     uint16_t nameLen;
740     char *name;
741     uint32_t treeBitsLen;
742 } HuffHeader;
743
744 void freeHeader(HuffHeader *h) {
745     if(!h) return;
746     free(h->name);
747     h->name = NULL;
748 }
749
750 /**
751  * Now the encoded file is to be read and we want to get
752  * a one-to-one copy of the original file
753  *
754  * First header is read in the same order like it was written
755  *
756  * MAGIC BYTES WHICH IDENTIFY HUFF FILE FORMAT
757  * ORIGINAL SIZE OF THE FILE
758  * LENGTH OF ORIGINAL FILE NAME
759  * THE ORIGINAL FILE NAME
760  * THE SERIALIZED HUFFMAN TREE
761  */
762
763 Status readHeader(FILE *in, HuffHeader *h) {
764     memset(h, 0, sizeof(*h));
765
766     uint8_t magic[8];
767     const uint8_t magic_expected[] = {'H', 'U', 'F', 'F', '2', '4', '1', '2'};
768
769     if(fread(magic, 1, 8, in) != 8) return ERR_READ;
770     if(memcmp(magic, magic_expected, 8) != 0) return ERR_FORMAT;
771
772     if(fread(&h->originalSize, sizeof(h->originalSize), 1, in) != 1) return ERR_READ;
773     if(fread(&h->nameLen, sizeof(h->nameLen), 1, in) != 1) return ERR_READ;
774
775     h->name = (char*) malloc((size_t) h->nameLen + 1);
776     if(!h->name) return ERR_MEMORY;
777
778     if(h->nameLen > 0) {
779         if(fread(h->name, 1, h->nameLen, in) != h->nameLen) return ERR_READ;
780     }
781     h->name[h->nameLen] = '\0';
782
783     if(fread(&h->treeBitsLen, sizeof(h->treeBitsLen), 1, in) != 1) return ERR_READ;
784
785     return SUCCESS;
786 }
787
788 /*===== BIT READER =====*/
789
790 typedef struct {
791     FILE *f;
792     uint8_t buffer;
793     uint8_t bitPos;
794 } BitReader;
795

```

```

796 void initBitReader(BitReader *br, FILE *f) {
797     br->f = f;
798     br->buffer = 0;
799     br->bitPos = 0;
800 }
801
802 /**
803  * Reads a byte from the file and flushes it in the buffer
804  *
805  * Next read the buffer bit by bit and update the bits read
806  * in bitPos member of the BitReader struct
807  *
808  * Bits are read in MSB first order
809  */
810
811 Status readBit(BitReader *br, uint8_t *outBit) {
812     if(br->bitPos == 0) {
813         if(fread(&br->buffer, 1, 1, br->f) != 1) return ERR_READ;
814         br->bitPos = 8;
815     }
816
817     *outBit = (br->buffer >> (br->bitPos - 1)) & 1;
818     br->bitPos--;
819     return SUCCESS;
820 }
821
822 /**
823  * Multiple bits are read simultaneously and stored in
824  * unsigned 64 bit integer
825  *
826  * Bits are read one at a time using readBit() and combined
827  * into final value using left shift operation
828  */
829
830 Status readBitsU64(BitReader *br, uint32_t bitCount, uint64_t *outValue) {
831     uint64_t value = 0;
832
833     for(uint32_t bitIdx = 0; bitIdx < bitCount; bitIdx++) {
834         uint8_t bit = 0;
835
836         Status s = readBit(br, &bit);
837         if(s != SUCCESS) return s;
838
839         value = (value << 1) | (uint64_t) bit;
840     }
841
842     *outValue = value;
843     return SUCCESS;
844 }
845
846 // discard remaining bits in current byte
847 void alignToByte(BitReader *br) {
848     br->bitPos = 0;
849 }
850
851 /**
852  * Reads a Huffman tree in the same preorder bit format as writeTree(), but enforces
853  * a hard limit: it will consume exactly treeBitsLen bits overall.
854  *
855  * bitsLeft:
856  *   input = how many tree bits remain to be read
857  *   output = decremented as bits are consumed; should reach 0 when tree is fully read
858  */
859
860 Status readTree(BitReader *br, uint32_t *bitsLeft, Node **outNode) {
861     *outNode = NULL;
862
863     if(!*bitsLeft || !*bitsLeft) return ERR_FORMAT;

```

```

864
865     uint8_t tag;
866     Status s = readBit(br, &tag);
867     if(s != SUCCESS) return s;
868     (*bitsLeft)--;
869
870     if(tag == 1) {
871         // leaf: next 8 bits are byte value
872         if(*bitsLeft < 8) return ERR_FORMAT;
873
874         uint64_t byte = 0;
875         s = readBitsU64(br, 8, &byte);
876         if(s != SUCCESS) return s;
877         (*bitsLeft) -= 8;
878
879         s = createNode(outNode, (uint8_t) byte, 0, NULL, NULL);
880         if(s != SUCCESS) return s;
881
882         return SUCCESS;
883     }
884
885     // internal: read left subtree then right subtree
886     Node *left = NULL;
887     Node *right = NULL;
888
889     s = readTree(br, bitsLeft, &left);
890     if(s != SUCCESS) {
891         freeTree(&left);
892         return s;
893     }
894
895     s = readTree(br, bitsLeft, &right);
896     if(s != SUCCESS) {
897         freeTree(&left);
898         freeTree(&right);
899         return s;
900     }
901
902     s = createNode(outNode, 0, 0, left, right);
903     if(s != SUCCESS) {
904         freeTree(&left);
905         freeTree(&right);
906         return s;
907     }
908
909     return SUCCESS;
910 }
911
912 /*===== DECODER =====*/
913
914 /**
915  * Flush count bits into the output file
916  */
917
918 Status flushBuffer(FILE *out, uint8_t *buffer, size_t *count) {
919     if(*count == 0) return SUCCESS;
920
921     if(fwrite(buffer, 1, *count, out) != *count) return ERR_WRITE;
922     *count = 0;
923     return SUCCESS;
924 }
925
926 /**
927  * Handles the edge case first
928  * if the file contained only one unique byte then it is written
929  * with that byte until original size bytes are written
930  *
931  * Bits are read from the file and are used to traverse the tree

```

```

932  * until a leaf node is encountered whose bytes are then first stored
933  * in a buffer and then flushed to output file when buffer is full
934  *
935  * This operation continues till original size bytes are written
936  */
937
938  Status decodePayload(BitReader *br, FILE *out, const Node *root, uint64_t originalSize) {
939      if(!root) return ERR_FORMAT;
940
941      Status s = SUCCESS;
942      uint8_t *buffer = NULL;
943      size_t outCount = 0;
944
945      buffer = (uint8_t*) malloc(BUFFER_SIZE);
946      if(!buffer) return ERR_MEMORY;
947
948      // special case: single byte
949      if(!root->left && !root->right) {
950          for(uint64_t i=0; i<originalSize; i++) {
951              buffer[outCount++] = root->byte;
952
953              if(outCount == BUFFER_SIZE) {
954                  s = flushBuffer(out, buffer, &outCount);
955                  if(s != SUCCESS) goto cleanup;
956              }
957
958              s = flushBuffer(out, buffer, &outCount);
959              if(s != SUCCESS) goto cleanup;
960
961              s = SUCCESS;
962              goto cleanup;
963          }
964      }
965
966      uint64_t written = 0;
967      const Node *curr = root;
968
969      while(written < originalSize) {
970          uint8_t bit = 0;
971
972          s = readBit(br, &bit);
973          if(s != SUCCESS) goto cleanup;
974
975          curr = bit ? curr->right : curr->left;
976          if(!curr) {
977              s = ERR_FORMAT;
978              goto cleanup;
979          }
980
981          if(!curr->left && !curr->right) {
982              buffer[outCount++] = curr->byte;
983              written++;
984              curr = root;
985
986              if(outCount == BUFFER_SIZE) {
987                  s = flushBuffer(out, buffer, &outCount);
988                  if(s != SUCCESS) goto cleanup;
989              }
990          }
991      }
992
993      s = flushBuffer(out, buffer, &outCount);
994      if(s != SUCCESS) goto cleanup;
995
996      cleanup:
997          free(buffer);
998          return s;
999  }

```

```

1000
1001  /*===== UTILITIES =====*/
1002
1003  /**
1004   * Cleanup function to provide a single path for the program
1005   * to terminate in case of an error or successful run
1006   */
1007
1008  void cleanupAll(uint64_t *freq, MinHeap *h, Node *root, Code *table, char *outName) {
1009      free(freq);
1010
1011      if(root) {
1012          freeTree(&root); // frees all nodes
1013          destroyHeapStruct(&h); // heap contains stale pointers, but only struct is freed
1014      }
1015
1016      else {
1017          destroyHeapWithNodes(&h); // frees nodes when tree not formed
1018      }
1019
1020      free(table);
1021      free(outName);
1022  }
1023
1024  void printCodes(const Code *table) {
1025      for (int byte = 0; byte < 256; byte++) {
1026          if (table[byte].length == 0) continue;
1027
1028          printf("%c: ", byte);
1029
1030          uint64_t bits = table[byte].bits;
1031          uint8_t length = table[byte].length;
1032
1033          for (uint8_t i = length; i > 0; i--) {
1034              putchar(((bits >> (i - 1)) & 1) ? '1' : '0');
1035          }
1036
1037          putchar('\n');
1038      }
1039  }
1040
1041  /**
1042   * Helper function to print the status codes in case of an error
1043   * or SUCCESS
1044   */
1045
1046  void printStatus(const char *testName, Status s) {
1047      fprintf(stderr, "%s: ", testName);
1048      switch (s) {
1049          case SUCCESS:
1050              fprintf(stderr, "[SUCCESS]\n");
1051              break;
1052
1053          case ERR_FILE:
1054              fprintf(stderr, "[ERR_FILE]\n");
1055              break;
1056
1057          case ERR_MEMORY:
1058              fprintf(stderr, "[ERR_MEMORY]\n");
1059              break;
1060
1061          case ERR_READ:
1062              fprintf(stderr, "[ERR_READ]\n");
1063              break;
1064
1065          case ERR_RECURSION_DEPTH:
1066              fprintf(stderr, "[ERR_RECURSION_DEPTH]\n");
1067              break;

```

```

1068
1069     case ERR_HEAP_CAPACITY_FULL:
1070         fprintf(stderr, "[ERR_HEAP_CAPACITY_FULL]\n");
1071         break;
1072
1073     case ERR_WRITE:
1074         fprintf(stderr, "[ERR_WRITE]\n");
1075         break;
1076
1077     case ERR_FORMAT:
1078         fprintf(stderr, "[ERR_FORMAT]\n");
1079         break;
1080
1081     case ERR_ABORTED:
1082         fprintf(stderr, "[ERR_ABORTED]\n");
1083         break;
1084     }
1085 }
1086
1087 /**
1088  * Helper function to generate the output name for the encoded file
1089  */
1090
1091 Status makeOutputName(char **outName, const char *inputPath) {
1092     *outName = NULL;
1093
1094     const char *base = baseName(inputPath);
1095     const char *ext = ".huff";
1096
1097     size_t baseLen = strlen(base);
1098     size_t extLen = strlen(ext);
1099
1100     *outName = (char*) malloc(baseLen + extLen + 1);
1101     if(!*outName) return ERR_MEMORY;
1102
1103     memcpy(*outName, base, baseLen);
1104     memcpy(*outName + baseLen, ext, extLen);
1105     (*outName)[baseLen + extLen] = '\0';
1106
1107     return SUCCESS;
1108 }
1109
1110 /*===== CMDLINE ARGS PARSING =====*/
1111
1112 /**
1113  * Helper functions to process the command line arguments and
1114  * handle the case where the file already exists in the current directory
1115  */
1116
1117 typedef enum {
1118     NONE,
1119     COMPRESS,
1120     DECOMPRESS
1121 } Mode;
1122
1123 Status parseArgs(int argc, char **argv, Mode *mode, const char **inputPath) {
1124     *mode = NONE;
1125     *inputPath = NULL;
1126
1127     if(argc != 3) return ERR_FILE;
1128
1129     if(strcmp(argv[1], "-c") == 0) {
1130         *mode = COMPRESS;
1131         *inputPath = argv[2];
1132         return SUCCESS;
1133     }
1134
1135     else if(strcmp(argv[1], "-x") == 0) {

```

```

1136         *mode = DECOMPRESS;
1137         *inputPath = argv[2];
1138         return SUCCESS;
1139     }
1140
1141     return ERR_FILE;
1142 }
1143
1144 char fileExists(const char *path) {
1145     FILE *f = fopen(path, "rb");
1146     if(f) {
1147         fclose(f);
1148         return 1;
1149     }
1150
1151     return 0;
1152 }
1153
1154 int promptOverwrite(const char *path) {
1155     fprintf(stderr, "File '%s' already exists. Overwrite? [y/N]: ", path);
1156     fflush(stderr);
1157
1158     int c = getchar();
1159     while(c != '\n' && c != EOF) {
1160         int d = getchar();
1161         if(d == '\n' || d == EOF) break;
1162     }
1163
1164     return (c == 'y' || c == 'Y');
1165 }
1166
1167 /*===== HIGH-LEVEL FILE OPS =====*/
1168
1169 /**
1170  * These are high-level wrapper functions that just call the necessary functions
1171  * in order to achieve the desired task
1172  *
1173  * First the file is read to make the freq table and get the total bytes
1174  * then the heap is built
1175  *
1176  * Huffman tree is generated using heap
1177  *
1178  * Table to store the huffman codes is initialized and
1179  * huffman codes are produced by traversing the tree
1180  * built in the previous steps
1181  *
1182  * The file is then encoded by the encoder pipeline
1183  */
1184
1185 Status compressFile(const char *inputPath) {
1186     Status s = SUCCESS;
1187
1188     uint64_t *freq = NULL;
1189     MinHeap *heap = NULL;
1190     Node *root = NULL;
1191     uint64_t totalBytes = 0;
1192     Code *table = NULL;
1193     char *outName = NULL;
1194
1195     s = readFile(inputPath, &freq, &totalBytes);
1196     if(s != SUCCESS) goto cleanup;
1197
1198     if(totalBytes == 0) {
1199         s = ERR_FILE; // or ERR_FORMAT if you prefer
1200         goto cleanup;
1201     }
1202
1203     s = buildHeap(&heap, freq);

```

```

1204     if(s != SUCCESS) goto cleanup;
1205
1206     s = buildHuffmanTree(heap, &root);
1207     if(s != SUCCESS) goto cleanup;
1208
1209     s = initTable(&table);
1210     if(s != SUCCESS) goto cleanup;
1211
1212     s = buildCodes(root, table, 0, 0);
1213     if(s != SUCCESS) goto cleanup;
1214
1215     s = makeOutputName(&outName, inputPath);
1216     if(s != SUCCESS) goto cleanup;
1217
1218     s = encodeFile(inputPath, outName, table, totalBytes, root);
1219     if(s != SUCCESS) goto cleanup;
1220
1221     printf("Original File Size: %" PRIu64 " Bytes\n", totalBytes);
1222     printf("Wrote Compressed File: %s\n", outName);
1223
1224     cleanup:
1225         cleanupAll(freq, heap, root, table, outName);
1226         return s;
1227 }
1228
1229 /**
1230  * First the header is read
1231  * Then the actual bits are read and converted to the
1232  * original file
1233  */
1234
1235 Status decodeFile(const char *inHuffPath) {
1236     Status s = SUCCESS;
1237     FILE *in = NULL;
1238     FILE *out = NULL;
1239
1240     HuffHeader header;
1241     memset(&header, 0, sizeof(header));
1242
1243     Node *root = NULL;
1244
1245     in = fopen(inHuffPath, "rb");
1246     if(!in) {
1247         s = ERR_FILE;
1248         goto cleanup;
1249     }
1250
1251     s = readHeader(in, &header);
1252     if(s != SUCCESS) goto cleanup;
1253
1254     if(header.treeBitsLen == 0) {
1255         s = ERR_FORMAT;
1256         goto cleanup;
1257     }
1258
1259     if(fileExists(header.name)) {
1260         if(!promptOverwrite(header.name)) {
1261             s = ERR_ABORTED;
1262             goto cleanup;
1263         }
1264     }
1265
1266     out = fopen(header.name, "wb");
1267     if(!out) {
1268         s = ERR_FILE;
1269         goto cleanup;
1270     }
1271

```

```

1272     BitReader br;
1273     initBitReader(&br, in);
1274
1275     uint32_t bitsLeft = header.treeBitsLen;
1276     s = readTree(&br, &bitsLeft, &root);
1277     if(s != SUCCESS) goto cleanup;
1278
1279     if(bitsLeft != 0) {
1280         s = ERR_FORMAT;
1281         goto cleanup;
1282     }
1283
1284     alignToByte(&br);
1285
1286     s = decodePayload(&br, out, root, header.originalSize);
1287     if(s != SUCCESS) goto cleanup;
1288
1289     printf("Original File Size: %" PRIu64 " Bytes\n", header.originalSize);
1290     printf("Wrote Decoded File: %s\n", header.name);
1291
1292     cleanup:
1293         if(out) fclose(out);
1294         if(in) fclose(in);
1295         freeTree(&root);
1296         freeHeader(&header);
1297         return s;
1298     }
1299
1300     /*===== MAIN =====*/
1301
1302     int main(int argc, char **argv) {
1303         Mode mode;
1304         const char *inputPath = NULL;
1305         Status s;
1306
1307         s = parseArgs(argc, argv, &mode, &inputPath);
1308         if(s != SUCCESS || mode == NONE) {
1309             fprintf(stderr, "Usage:\n");
1310             fprintf(stderr, " %s -c <input_file>      (compress)\n", argv[0]);
1311             fprintf(stderr, " %s -x <input_file>      (decompress)\n", argv[0]);
1312             return 1;
1313         }
1314
1315         if(mode == COMPRESS) s = compressFile(inputPath);
1316         else s = decodeFile(inputPath);
1317
1318         if(s != SUCCESS) {
1319             printStatus("ERROR", s);
1320         }
1321
1322         return (s == SUCCESS) ? 0 : 1;
1323     }

```